

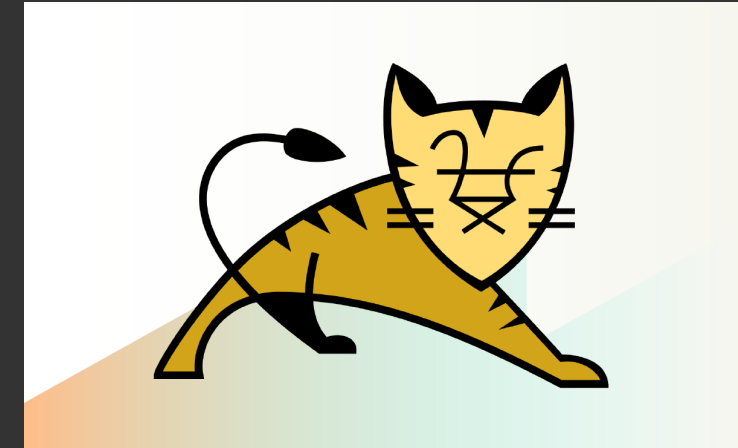


Async

Tomcat Server Threading Model

Tomcat Threading Model

Tomcat is a popular open source servlet container in the Java ecosystem. Tomcat is the default container for spring boot web applications (spring boot also supports other containers). The web server uses a thread pool (e.g., TomcatExecutor) to process incoming HTTP requests.

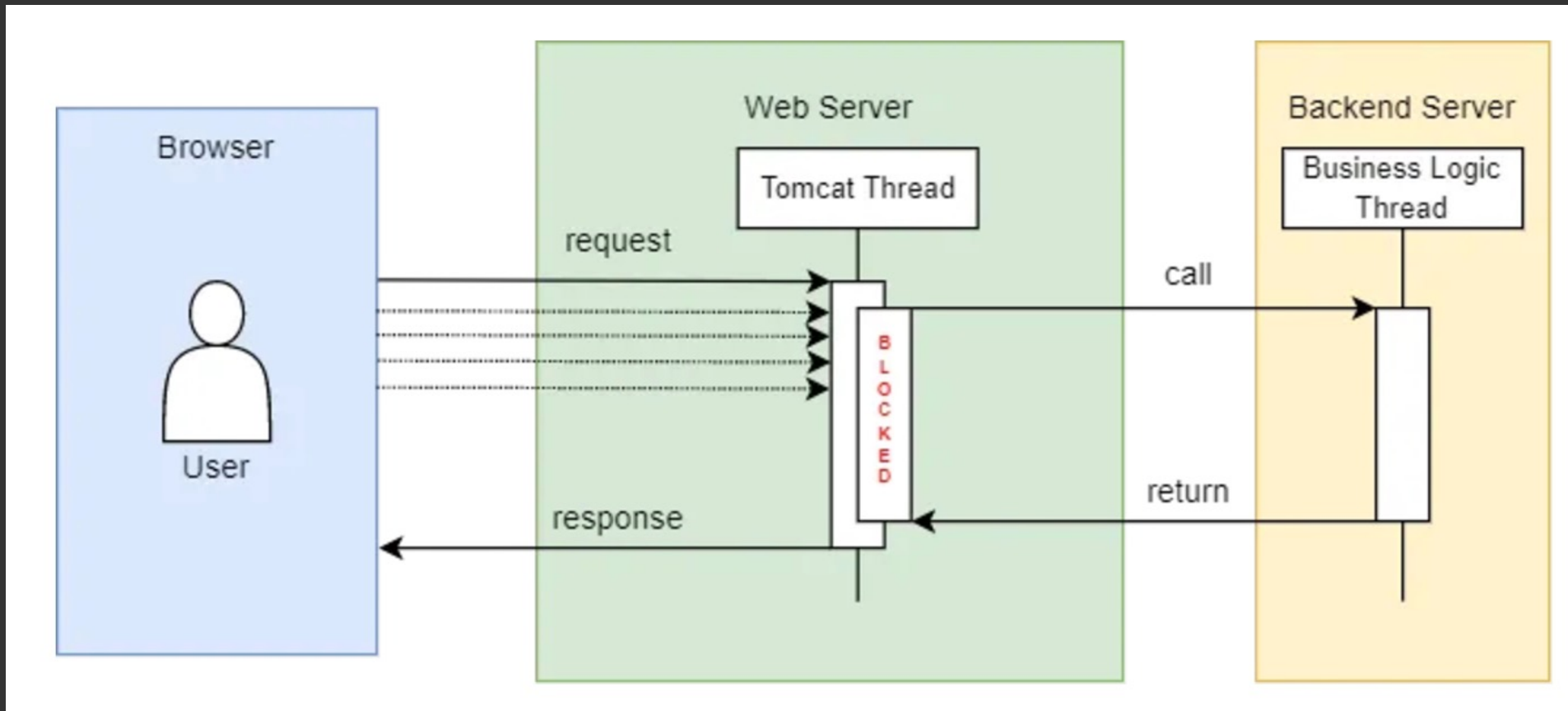


How it works:

1. When a request arrives, a thread is taken from **HTTP Thread Pool** (commonly referred to as the "worker threads").
2. The thread processes the request (e.g., executes the controller logic).
3. Once completed, the thread is returned to the pool.

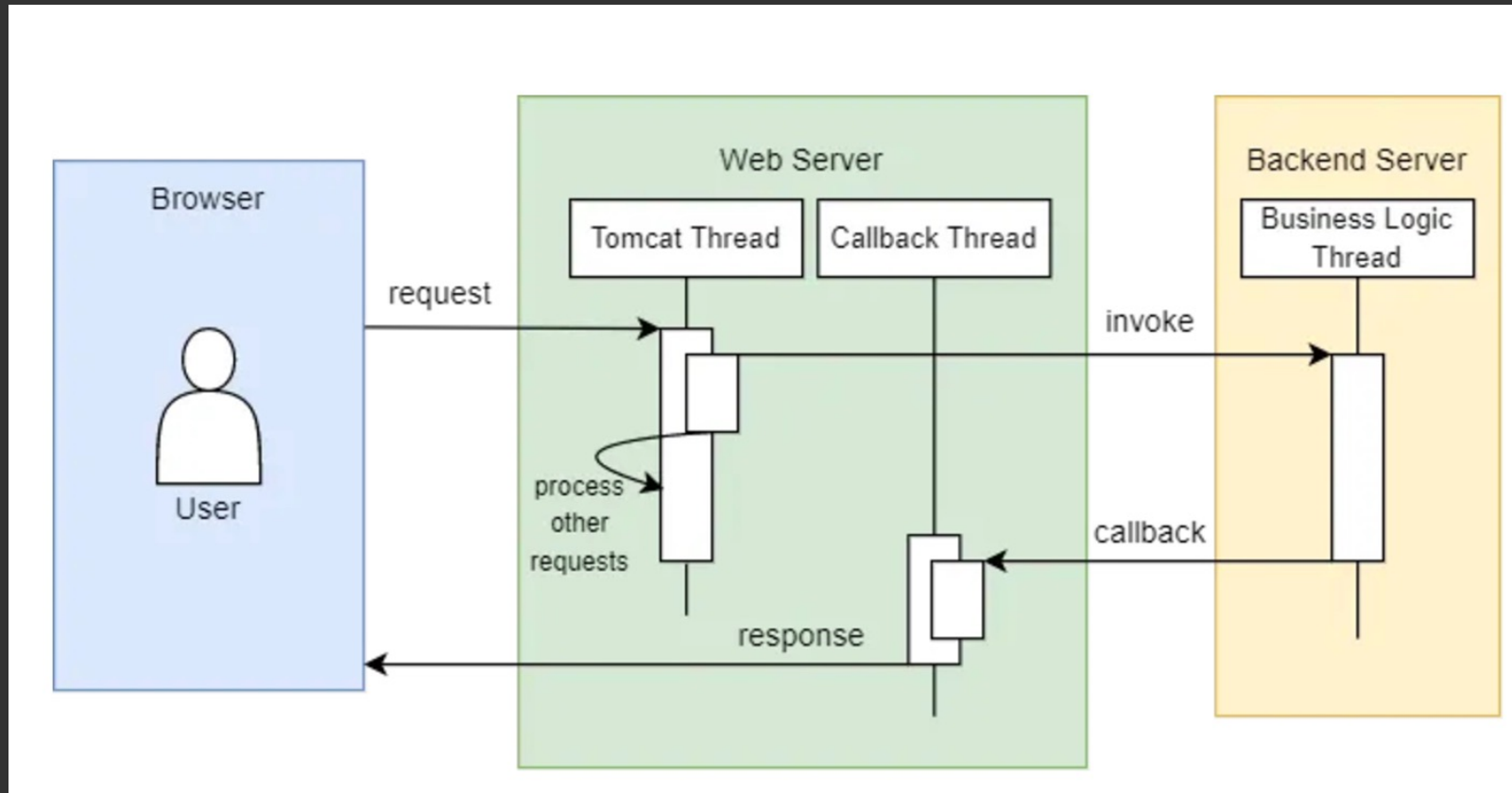
Tomcat Blocking Flow

If the application logic is slow (e.g., waiting on a database call or processing a long task), the thread remains blocked, unable to handle new requests.



Tomcat Async (Non-blocking) Flow

If the request involves an asynchronous operation (e.g., `@Async` or `DeferredResult`), the work is handed off to a callback thread (from an executor or thread pool). The Tomcat web server thread is freed up at this point to handle other requests.



Default Config for Tomcat

Default configurations for Tomcat:

- Maximum Threads: 200
- Minimum Idle Threads: 10
- Queue Size: Unbounded by default (can lead to memory issues if too many requests arrive).

You can customize this via application.properties:

```
server.tomcat.threads.max=200
```

```
server.tomcat.threads.min-spare=10
```

```
server.tomcat.accept-count=100 //When the queue exceeds this limit, new  
requests are rejected with a 503 Service Unavailable.
```

Spring Boot Thread Safety

Spring Beans default to Singleton scope, which means that there will only ever be one instance of them while your application is running. This means that all of your Controllers, Services, Repositories, etc code will be shared by multiple threads, which means they will be shared by multiple user requests running at the same time.

- You should be designing all of these components to be stateless.
- User state should only exist in parameters, and inside functions.
- It's totally fine for your Repository class to store state about the database url, username, and password. These values are not user-specific.

